



UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT



Data Structure and Algorithm

Laboratory Activity No. 12

Graph Searching Algorithm

Submitted by:
Gabijan, Rhovic M.

Instructor:
Engr. Maria Rizette H. Sayo

October 30, 2025

I. Objectives

Introduction

Depth-First Search (DFS)

- Explores as far as possible along each branch before backtracking
- Uses stack data structure (either explicitly or via recursion)
- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

Breadth-First Search (BFS)

- Explores all neighbors at current depth before moving deeper
- Uses queue data structure
- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

This laboratory activity aims to implement the principles and techniques in:

- Understand and implement Depth-First Search (DFS) and Breadth-First Search (BFS) algorithms
- Compare the traversal order and behavior of both algorithms
- Analyze time and space complexity differences

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Graph Implementation

```
from collections import deque
import time

class Graph:
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, vertex1, vertex2, directed=False):
        self.add_vertex(vertex1)
        self.add_vertex(vertex2)

        self.adj_list[vertex1].append(vertex2)
        if not directed:
            self.adj_list[vertex2].append(vertex1)
```

```

def display(self):
    for vertex, neighbors in self.adj_list.items():
        print(f"{vertex}: {neighbors}")

```

2. DFS Implementation

```

def dfs_recursive(graph, start, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(start)
    path.append(start)
    print(f"Visiting: {start}")

    for neighbor in graph.adj_list[start]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited, path)

    return path

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    path = []

    print("DFS Iterative Traversal:")
    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            path.append(vertex)
            print(f"Visiting: {vertex}")

            # Add neighbors in reverse order for same behavior as recursive
            for neighbor in reversed(graph.adj_list[vertex]):
                if neighbor not in visited:
                    stack.append(neighbor)
    return path

```

3. BFS Implementation

```

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    path = []

    print("BFS Traversal:")
    while queue:
        vertex = queue.popleft()
        if vertex not in visited:
            visited.add(vertex)
            path.append(vertex)
            print(f"Visiting: {vertex}")

```

```

    for neighbor in graph.adj_list[vertex]:
        if neighbor not in visited:
            queue.append(neighbor)

return path

```

Questions:

- 1 When would you prefer DFS over BFS and vice versa?
- 2 What is the space complexity difference between DFS and BFS?
- 3 How does the traversal order differ between DFS and BFS?
- 4 When does DFS recursive fail compared to DFS iterative?

III. Results

1. DFS is preferred when the goal is to explore deep paths since it quickly dives into deeper nodes before backtracking. It uses fewer nodes than BFS. On the other hand, BFS is preferred for situations where shortest path is needed. It is generally better for wide graphs, while DFS is for narrow.
2. Although both DFS and BFS have a worst-case space complexity of $O(V)$, their actual memory use can differ dramatically. Because DFS only needs to store the nodes along the current path plus a few neighbors, it usually utilizes a lot less memory, making it more effective for deep but narrow graphs. BFS, on the other hand, frequently needs more memory because it stores every node in a level simultaneously inside the queue. In practice, BFS requires more memory than DFS since the BFS queue can get quite huge for wide graphs.
3. DFS explores the graph as deeply as possible along on path before backtracking, which means its traversal is path-focused pattern. In contrast, BFS explore the graph level by level, visiting all immediate neighbors before moving to the next layer. This makes DFS dive vertically while BFS horizontally.
4. DFS implemented recursively can fail when the graph is very deep, because Python has a recursion depth limit that can lead to a `RecursionError` if too many recursive calls occur. It may also fail if the graph contains cycles and the visited set is not properly handled, causing infinite recursion. On the other hand, DFS iterative avoids these issues because it uses an explicit stack rather than relying on the system's call stack, allowing it to handle very deep or complex graphs without hitting recursion limits. As a result, DFS iterative is more robust for large-scale or deep graph structures.

IV. Conclusion

Understanding queues, along with the enqueue and dequeue operations, highlights the importance of organizing data in a way that supports smooth and efficient processing. These concepts are foundational in computer science because they allow programs to handle tasks in an orderly, predictable manner. Even in graphical frameworks like Kivy and KivyMD, where interfaces and interactive components are built, the logic behind data flow still relies on structured principles such as queues. Altogether, these topics show how essential data structures and clean program design are in developing responsive and reliable applications.

References

- [1] Co Arthur O.. “University of Caloocan City Computer Engineering Department Honor Code,” UCC-CpE Departmental Policies, 2020.